AGUTS

Ahmedabad Gandhinagar Unified Transit System

Milestone 3 — File 2 of 3

BCNF Normalization Proofs

IT-214 DBMS | Winter 2025-26 | DAU | Lab Group 5

BCNF Proofs for AGUTS Relations

Definition. A relation R is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency X → Y that holds in R, X is a superkey of R.

Proof strategy for each relation:

1. State the relation schema R and its attribute set U.
2. List the minimal set of non-trivial FDs that hold in R.
3. For each FD X → Y, compute the closure X^+ under those FDs.
4. Show that X^+ = U, proving X is a superkey.
5. Conclude BCNF.

Summary — All 26 Relations

#	Relation	Primary Key	Non-Trivial FDs	BCNF?
1	**CITY**	{city_id}	city_id → {city_name, state}	✓ Yes
2	**TRANSIT_OPERATOR**	{operator_id}	operator_id → {operator_name, service_type, city_id}	✓ Yes
3	**METRO_LINE**	{line_id}	line_id → {line_name, line_code, operator_id}	✓ Yes
4	**DEPOT**	{depot_id}	depot_id → {depot_name, area, pincode, operator_id}	✓ Yes
5	**VEHICLE**	{vehicle_id}	vehicle_id → {reg_no, capacity, mfr_year, mfr_name, mfr_info, status, operator_id, depot_id}	✓ Yes
6	**BUS**	{vehicle_id}	vehicle_id → {fuel_type, bus_type}	✓ Yes
7	**METRO_TRAIN**	{vehicle_id}	vehicle_id → {train_number, num_coaches}	✓ Yes
8	**MAINTENANCE**	{maintenance_id}	maintenance_id → {vehicle_id, start_date, end_date, description}	✓ Yes
9	**STOP**	{stop_id}	stop_id → {stop_name, stop_type, latitude, longitude, city_id, zone_id}	✓ Yes

10	**INTERCHANGE**	{interchange_id}	interchange_id → {interchange_type, description}	✓ Yes
11	**INTERCHANGE_STOP**	{interchange_id, stop_id}	∅ (key-only)	✓ Yes
12	**FARE_ZONE**	{zone_id}	zone_id → {zone_name}	✓ Yes
13	**ROUTE**	{route_id}	route_id → {route_number, route_name, distance_km}	✓ Yes
14	**ROUTE_STOP**	{route_id, sequence_no}	{route_id, sequence_no} → {stop_id}	✓ Yes
15	**PASSENGER**	{passenger_id}	passenger_id → {f_name, l_name, email, phone, dob, concession_type, city_id}	✓ Yes
16	**SMART_CARD**	{card_id}	card_id → {passenger_id, issue_date, balance, card_status, expiry_date}	✓ Yes
17	**STAFF**	{staff_id}	staff_id → {full_name, contact, doj, role, certificate, operator_id, depot_id}	✓ Yes
18	**DRIVER**	{staff_id}	staff_id → {license_no}	✓ Yes
19	**CONDUCTOR**	{staff_id}	staff_id → {badge_no}	✓ Yes
20	**MOTORMAN**	{staff_id}	staff_id → {metro_license_no}	✓ Yes
21	**TRIP**	{trip_id}	trip_id → {trip_date, etd, eta, status, delay, route_id, vehicle_id, staff_id}	✓ Yes
22	**TRIP_STOP**	{trip_id, stop_order}	{trip_id, stop_order} → {stop_id, sched_arrival, actual_arrival}	✓ Yes
23	**FARE**	{fare_id}	fare_id → {amount, concession_type, from_zone_id, to_zone_id}	✓ Yes
24	**PASS**	{pass_id}	pass_id → {pass_type, service_type, valid_from, valid_to, passenger_id, fare_id}	✓ Yes
25	**TICKET_TRANSACTION**	{txn_id}	txn_id → {ticket_type, amount, txn_dt, pay_mode, passenger_id, trip_id, card_id}	✓ Yes
26	**COMPLAINT**	{complaint_id}	complaint_id → {category, filed_dt, description, resolved_dt, status, passenger_id, trip_id}	✓ Yes

Detailed BCNF Proofs — Each Relation

Proof 1: CITY

```
U = {city_id, city_name, state}
```

Functional Dependencies:

```
city_id → city_name

city_id → state
```

Closure computation for $\{city\_id\}^+$:

```
Apply city_id → city_name and city_id → state

{city_id}⁺ = {city_id, city_name, state} = U
```

city_id is a superkey. The only non-trivial FD has city_id on its LHS. ✓ **BCNF**

Proof 2: TRANSIT_OPERATOR

```
U = {operator_id, operator_name, service_type, city_id}
```

Functional Dependencies:

```
operator_id → operator_name

operator_id → service_type

operator_id → city_id
```

Closure computation for $\{operator\_id\}^+$:

```
Apply all three FDs

{operator_id}⁺ = {operator_id, operator_name, service_type, city_id} = U
```

operator_id is a superkey. All FD determinants are operator_id. ✓ **BCNF**

Proof 3: METRO_LINE

```
U = {line_id, line_name, line_code, operator_id}
```

Functional Dependencies:

```
line_id → line_name

line_id → line_code

line_id → operator_id
```

Closure computation for {line_id}⁺:

```
{line_id}⁺ = {line_id, line_name, line_code, operator_id} = U
```

line_id is a superkey. ✓ **BCNF**

Proof 4: DEPOT

```
U = {depot_id, depot_name, area, pincode, operator_id}
```

Functional Dependencies:

```
depot_id → depot_name

depot_id → area

depot_id → pincode

depot_id → operator_id
```

Closure computation for {depot_id}⁺:

```
{depot_id}⁺ = {depot_id, depot_name, area, pincode, operator_id} = U
```

depot_id is a superkey. ✓ **BCNF**

Proof 5: VEHICLE

```
U = {vehicle_id, registration_no, capacity, manufacture_year,
manufacturer_name, manufacturer_info, status, operator_id, depot_id}
```

Functional Dependencies:

```
vehicle_id → registration_no

vehicle_id → capacity

vehicle_id → manufacture_year

vehicle_id → manufacturer_name

vehicle_id → manufacturer_info

vehicle_id → status

vehicle_id → operator_id

vehicle_id → depot_id
```

Closure computation for {vehicle_id}⁺:

```
{vehicle_id}⁺ = {vehicle_id, registration_no, capacity, manufacture_year,

  manufacturer_name, manufacturer_info, status, operator_id, depot_id} = U
```

vehicle_id is a superkey. ✓ **BCNF**

Proof 6: BUS

```
U = {vehicle_id, fuel_type, bus_type}
```

Functional Dependencies:

```
vehicle_id → fuel_type

vehicle_id → bus_type
```

Closure computation for {vehicle_id}⁺:

```
{vehicle_id}⁺ = {vehicle_id, fuel_type, bus_type} = U
```

vehicle_id is a superkey (inherited ISA PK from VEHICLE). ✓ **BCNF**

Proof 7: METRO_TRAIN

```
U = {vehicle_id, train_number, num_coaches}
```

Functional Dependencies:

```
vehicle_id → train_number

vehicle_id → num_coaches
```

Closure computation for {vehicle_id}⁺:

```
{vehicle_id}⁺ = {vehicle_id, train_number, num_coaches} = U
```

vehicle_id is a superkey. ✓ **BCNF**

Proof 8: MAINTENANCE

```
U = {maintenance_id, vehicle_id, start_date, end_date, description}
```

Functional Dependencies:

```
maintenance_id → vehicle_id

maintenance_id → start_date

maintenance_id → end_date

maintenance_id → description
```

Closure computation for {maintenance_id}⁺:

```
{maintenance_id}⁺ = {maintenance_id, vehicle_id, start_date, end_date,
description} = U
```

maintenance_id is a superkey. ✓ **BCNF**

Proof 9: STOP

```
U = {stop_id, stop_name, stop_type, latitude, longitude, city_id, zone_id}
```

Functional Dependencies:

```
stop_id → stop_name

stop_id → stop_type

stop_id → latitude

stop_id → longitude

stop_id → city_id

stop_id → zone_id
```

Closure computation for {stop_id}⁺:

```
{stop_id}⁺ = {stop_id, stop_name, stop_type, latitude, longitude, city_id,
zone_id} = U
```

stop_id is a superkey. ✓ **BCNF**

Proof 10: INTERCHANGE

```
U = {interchange_id, interchange_type, description}
```

Functional Dependencies:

```
interchange_id → interchange_type

interchange_id → description
```

Closure computation for {interchange_id}⁺:

```
{interchange_id}⁺ = {interchange_id, interchange_type, description} = U
```

interchange_id is a superkey. ✓ **BCNF**

Proof 11: INTERCHANGE_STOP

```
U = {interchange_id, stop_id}
```

No non-trivial FDs (key-only relation).

Closure computation for N/A:

```
No non-trivial FDs exist. All attributes form the composite primary key.
```

A relation with no non-trivial FDs is trivially in BCNF. ✓ **BCNF**

Proof 12: FARE_ZONE

```
U = {zone_id, zone_name}
```

Functional Dependencies:

```
zone_id → zone_name
```

Closure computation for {zone_id}⁺:

```
{zone_id}⁺ = {zone_id, zone_name} = U
```

zone_id is a superkey. ✓ **BCNF**

Proof 13: ROUTE

```
U = {route_id, route_number, route_name, distance_km}
```

Functional Dependencies:

```
route_id → route_number

route_id → route_name

route_id → distance_km
```

Closure computation for {route_id}⁺:

```
{route_id}⁺ = {route_id, route_number, route_name, distance_km} = U
```

route_id is a superkey. ✓ **BCNF**

Proof 14: ROUTE_STOP

```
U = {route_id, sequence_no, stop_id}
```

Functional Dependencies:

```
{route_id, sequence_no} → stop_id
```

Closure computation for {route_id, sequence_no}⁺:

```
Apply {route_id, sequence_no} → stop_id

{route_id, sequence_no}⁺ = {route_id, sequence_no, stop_id} = U
```

{route_id, sequence_no} is the composite primary key and therefore a superkey. ✓ **BCNF**

Proof 15: PASSENGER

```
U = {passenger_id, f_name, l_name, email, phone, date_of_birth,
concession_type, city_id}
```

Functional Dependencies:

```
passenger_id → f_name

passenger_id → l_name

passenger_id → email

passenger_id → phone

passenger_id → date_of_birth

passenger_id → concession_type

passenger_id → city_id
```

Closure computation for {passenger_id}⁺:

```
{passenger_id}⁺ = {passenger_id, f_name, l_name, email, phone,
date_of_birth, concession_type, city_id} = U
```

passenger_id is a superkey. ✓ **BCNF**

Proof 16: SMART_CARD

```
U = {card_id, passenger_id, issue_date, balance, card_status, expiry_date}
```

Functional Dependencies:

```
card_id → passenger_id

card_id → issue_date

card_id → balance

card_id → card_status

card_id → expiry_date
```

Closure computation for {card_id}⁺:

```
{card_id}⁺ = {card_id, passenger_id, issue_date, balance, card_status,
expiry_date} = U
```

card_id is a superkey. ✓ **BCNF**

Proof 17: STAFF

```
U = {staff_id, full_name, contact, date_of_joining, role, certificate,
operator_id, depot_id}
```

Functional Dependencies:

```
staff_id → full_name

staff_id → contact

staff_id → date_of_joining

staff_id → role

staff_id → certificate

staff_id → operator_id

staff_id → depot_id
```

Closure computation for {staff_id}⁺:

```
{staff_id}⁺ = {staff_id, full_name, contact, date_of_joining, role,
certificate, operator_id, depot_id} = U
```

staff_id is a superkey. ✓ **BCNF**

Proof 18: DRIVER

```
U = {staff_id, license_no}
```

Functional Dependencies:

```
staff_id → license_no
```

Closure computation for {staff_id}⁺:

```
{staff_id}⁺ = {staff_id, license_no} = U
```

staff_id is a superkey (ISA PK from STAFF). ✓ **BCNF**

Proof 19: CONDUCTOR

```
U = {staff_id, badge_no}
```

Functional Dependencies:

```
staff_id → badge_no
```

Closure computation for {staff_id}⁺:

```
{staff_id}⁺ = {staff_id, badge_no} = U
```

staff_id is a superkey. ✓ **BCNF**

Proof 20: MOTORMAN

```
U = {staff_id, metro_license_no}
```

Functional Dependencies:

```
staff_id → metro_license_no
```

Closure computation for {staff_id}⁺:

```
{staff_id}⁺ = {staff_id, metro_license_no} = U
```

staff_id is a superkey. ✓ **BCNF**

Proof 21: TRIP

```
U = {trip_id, trip_date, etd, eta, trip_status, delay_minutes, route_id,
vehicle_id, staff_id}
```

Functional Dependencies:

```
trip_id → trip_date

trip_id → etd

trip_id → eta

trip_id → trip_status

trip_id → delay_minutes

trip_id → route_id

trip_id → vehicle_id

trip_id → staff_id
```

Closure computation for {trip_id}⁺:

```
{trip_id}⁺ = {trip_id, trip_date, etd, eta, trip_status, delay_minutes,
route_id, vehicle_id, staff_id} = U
```

trip_id is a superkey. ✓ **BCNF**

Proof 22: TRIP_STOP

```
U = {trip_id, stop_order, stop_id, scheduled_arrival, actual_arrival}
```

Functional Dependencies:

```
{trip_id, stop_order} → stop_id

{trip_id, stop_order} → scheduled_arrival

{trip_id, stop_order} → actual_arrival
```

Closure computation for {trip_id, stop_order}⁺:

```
Apply all three FDs

{trip_id, stop_order}⁺ = {trip_id, stop_order, stop_id, scheduled_arrival,
actual_arrival} = U
```

{trip_id, stop_order} is the composite PK and a superkey. ✓ **BCNF**

Proof 23: FARE

`U = {fare_id, amount, concession_type, from_zone_id, to_zone_id}`

Functional Dependencies:

```
fare_id → amount

fare_id → concession_type

fare_id → from_zone_id

fare_id → to_zone_id
```

Closure computation for {fare_id}⁺:

```
{fare_id}⁺ = {fare_id, amount, concession_type, from_zone_id, to_zone_id} =
U
```

fare_id is a superkey. ✓ **BCNF**

Proof 24: PASS

`U = {pass_id, pass_type, service_type, valid_from, valid_to, passenger_id, fare_id}`

Functional Dependencies:

```
pass_id → pass_type

pass_id → service_type

pass_id → valid_from

pass_id → valid_to

pass_id → passenger_id

pass_id → fare_id
```

Closure computation for {pass_id}⁺:

```
{pass_id}⁺ = {pass_id, pass_type, service_type, valid_from, valid_to,
passenger_id, fare_id} = U
```

pass_id is a superkey. ✓ **BCNF**

Proof 25: TICKET_TRANSACTION

```
U = {txn_id, ticket_type, amount, txn_datetime, payment_mode, passenger_id,
trip_id, card_id}
```

Functional Dependencies:

```
txn_id → ticket_type

txn_id → amount

txn_id → txn_datetime

txn_id → payment_mode

txn_id → passenger_id

txn_id → trip_id

txn_id → card_id
```

Closure computation for {txn_id}⁺:

```
{txn_id}⁺ = {txn_id, ticket_type, amount, txn_datetime, payment_mode,
passenger_id, trip_id, card_id} = U
```

txn_id is a superkey. ✓ **BCNF**

Proof 26: COMPLAINT

```
U = {complaint_id, category, filed_date, description, resolved_date, status,
passenger_id, trip_id}
```

Functional Dependencies:

```
complaint_id → category

complaint_id → filed_date

complaint_id → description

complaint_id → resolved_date

complaint_id → status

complaint_id → passenger_id

complaint_id → trip_id
```

Closure computation for {complaint_id}⁺:

```
{complaint_id}+ = {complaint_id, category, filed_date, description,
resolved_date, status, passenger_id, trip_id} = U
```

complaint_id is a superkey. ✓ **BCNF**